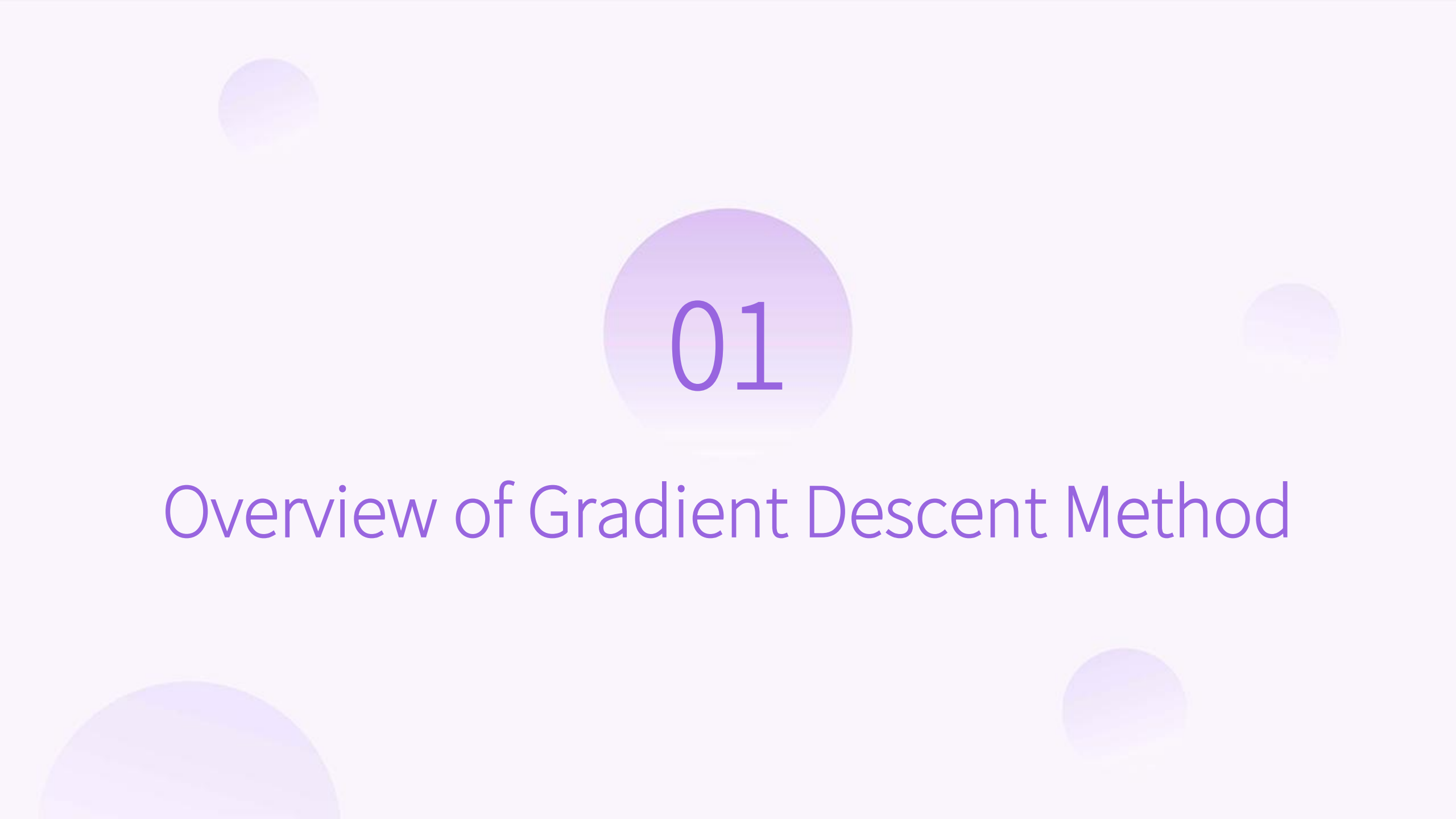


Neural Network Part 2

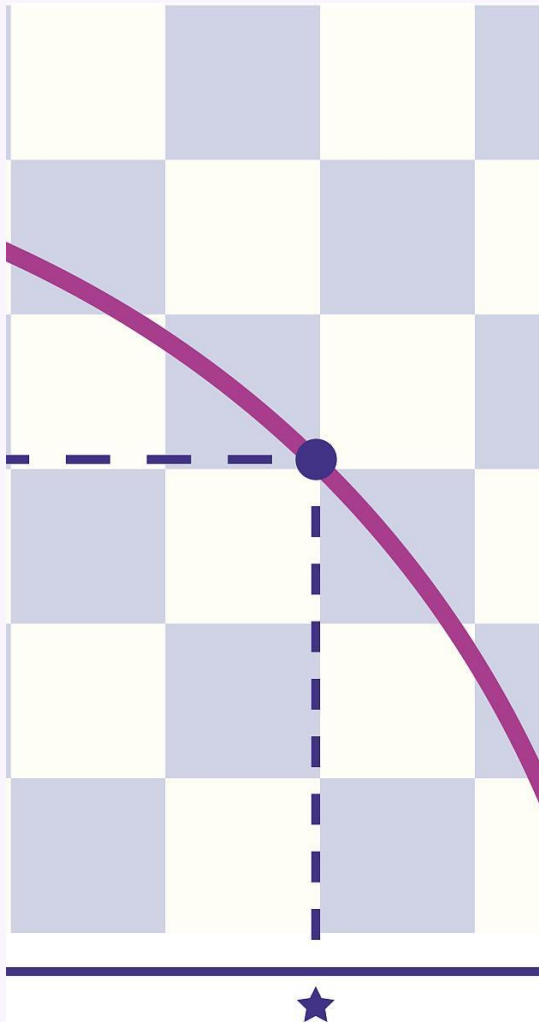
The background features several overlapping, semi-transparent purple circles of varying sizes. On the right side, there is a thin vertical purple line that extends from the bottom towards the top, ending in a small solid purple circle.



01

Overview of Gradient Descent Method

Basic principle of gradient descent



Gradient descent core

The gradient descent method iteratively adjusts parameters to reduce the loss function value until it converges to a local or global optimal solution.

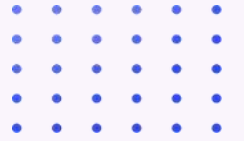
Descent direction

In each iteration, the gradient descent method updates parameters along the negative gradient direction of the current loss function surface to ensure that the loss continues to decrease.

The role of learning rate

The learning rate determines the step size of parameter updates. If it is too large, it may lead to missing the optimal solution, while if it is too small, it may result in slow convergence speed.

Gradient Descent Optimization



Gradient Descent Basics

Gradient descent updates weights in the opposite direction of the gradient to minimize the error. The learning rate controls the step size of each update.



Local Minima

Local minima are challenges in optimization where the algorithm gets stuck in suboptimal solutions. Techniques like momentum help escape these minima.



Adaptive Learning Rates

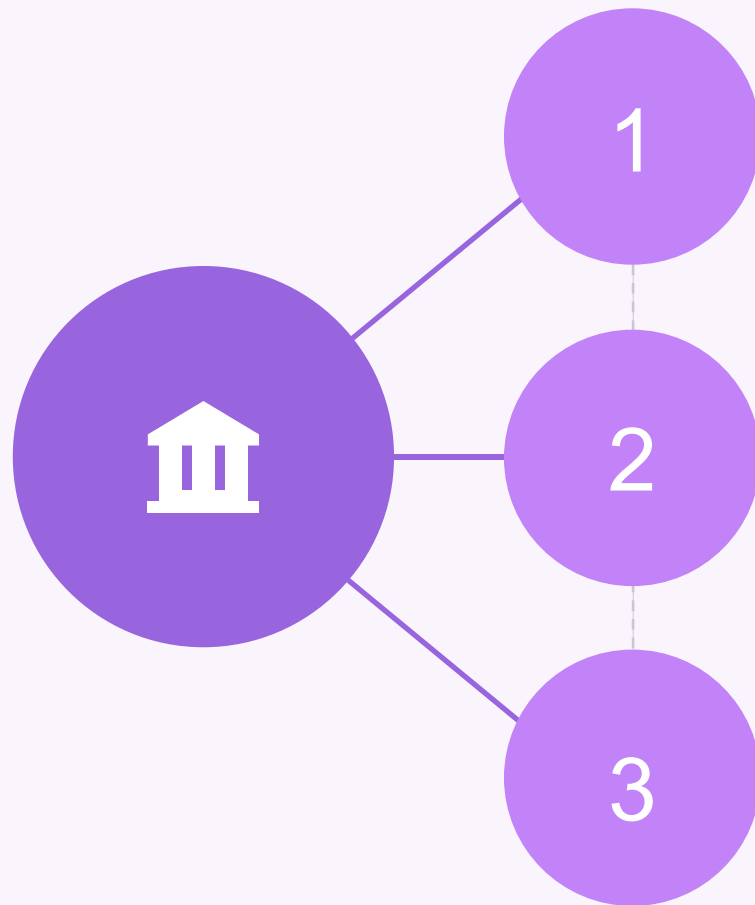
Adaptive learning rates adjust the step size dynamically, improving convergence speed and stability. Methods like Adam and RMSprop are commonly used.



Generalization

Proper optimization ensures that the network generalizes well to new data, balancing learning speed and stability.

Gradient Vector and Parameter Update



Gradient calculation

Gradient is a vector composed of partial derivatives of the loss function with respect to various parameters. It indicates the direction of the fastest growth of the loss function and is the key to parameter optimization.

Parameter update

Based on the calculated gradient, adjust the parameter values according to a certain learning rate to minimize the loss function, which is the core step of gradient descent method.

Feature scaling

When performing gradient descent optimization, scaling the features can help accelerate convergence speed and improve optimization efficiency, which is an important step in data processing.

Learning rate and convergence analysis

Importance of learning rate

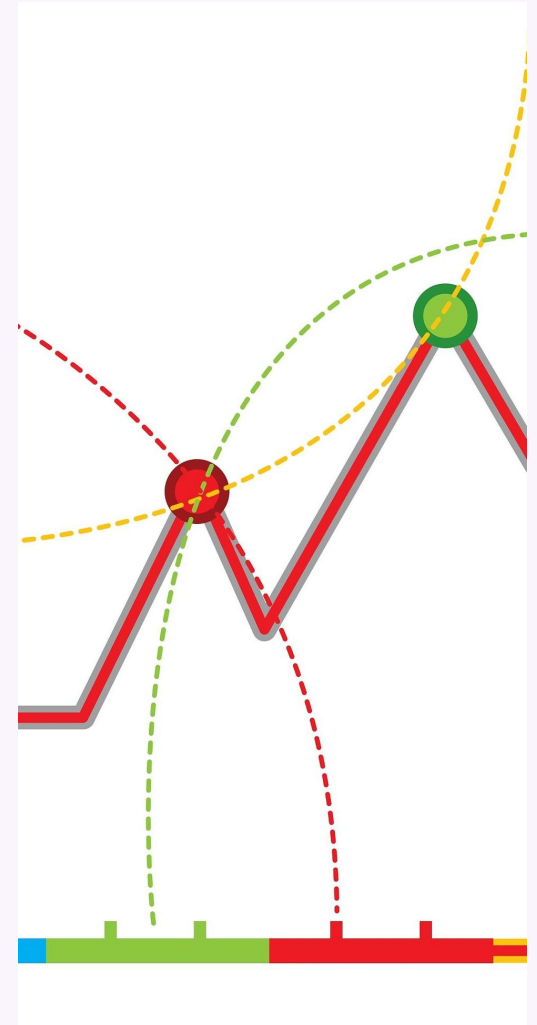
Learning rate is a key parameter in gradient descent algorithm, which controls the step size of parameter updates and has a significant impact on the convergence speed and stability of the algorithm.

Convergence analysis

In an ideal situation, gradient descent can linearly converge when the learning rate is moderate; When the learning rate is too high or low, it may cause the algorithm to not converge or converge too slowly.

Actual impact

The choice of learning rate has a significant impact on the optimization results and needs to be adjusted and optimized according to specific problems to ensure that the algorithm can converge to the optimal solution within a reasonable time.





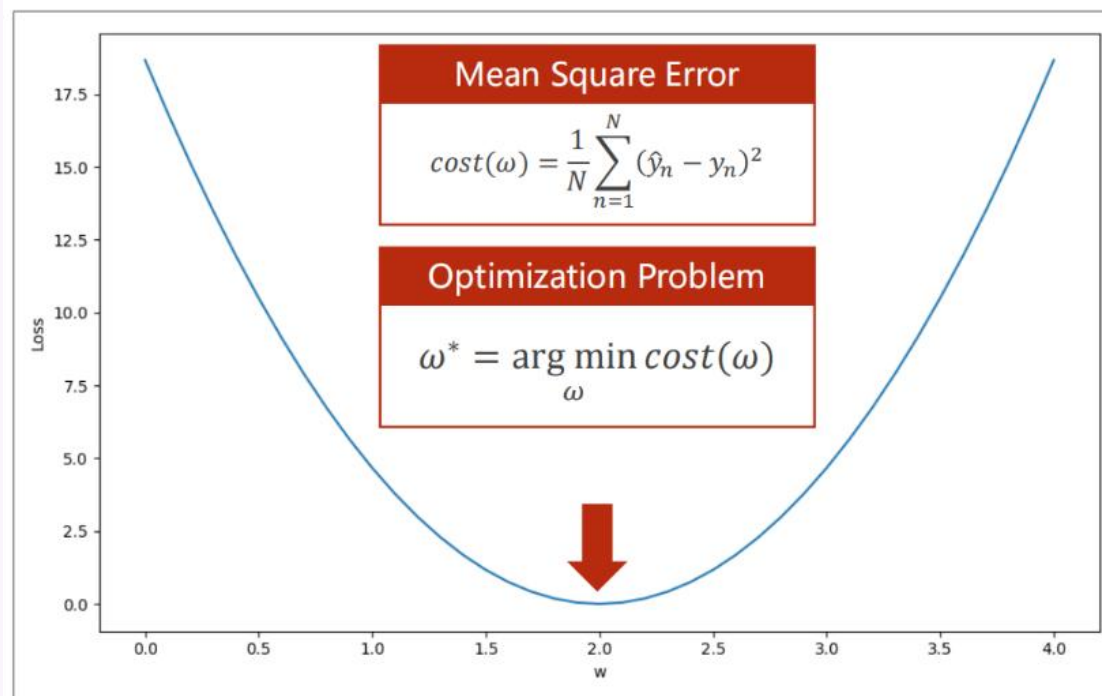
02

Gradient descent variant algorithm

The principle of batch gradient descent

- **Global gradient information:** During each parameter update, the batch gradient descent algorithm comprehensively considers all training samples, calculates global gradient information, and ensures that the update direction accurately reflects the downward trend of the overall loss function.
- **Convergence stability:** Batch gradient descent comprehensively considers all training samples and can converge more stably to the global or local optimal solution. It is suitable for small to medium-sized datasets and provides stable and reliable optimization results.
- **Computational resource requirements:** Batch gradient descent requires computing the gradients of all samples before each parameter update, which demands high computational resources. Especially when the dataset is large, it may result in a slow optimization process and high resource consumption.
- **Dealing with big datasets:** For big datasets, batch gradient descent may face the problem of insufficient computing resources, and its optimization efficiency is relatively low. Therefore, in situations where resources are limited, other optimization strategies or algorithms need to be considered.

Gradient descent optimization



Derivative

$$\begin{aligned} \frac{\partial cost(\omega)}{\partial \omega} &= \frac{\partial}{\partial \omega} \frac{1}{N} \sum_{n=1}^N (x_n \cdot \omega - y_n)^2 \\ &= \frac{1}{N} \sum_{n=1}^N \frac{\partial}{\partial \omega} (x_n \cdot \omega - y_n)^2 \\ &= \frac{1}{N} \sum_{n=1}^N 2 \cdot (x_n \cdot \omega - y_n) \frac{\partial (x_n \cdot \omega - y_n)}{\partial \omega} \\ &= \frac{1}{N} \sum_{n=1}^N 2 \cdot x_n \cdot (x_n \cdot \omega - y_n) \end{aligned}$$

Gradient

$$\frac{\partial cost}{\partial \omega}$$

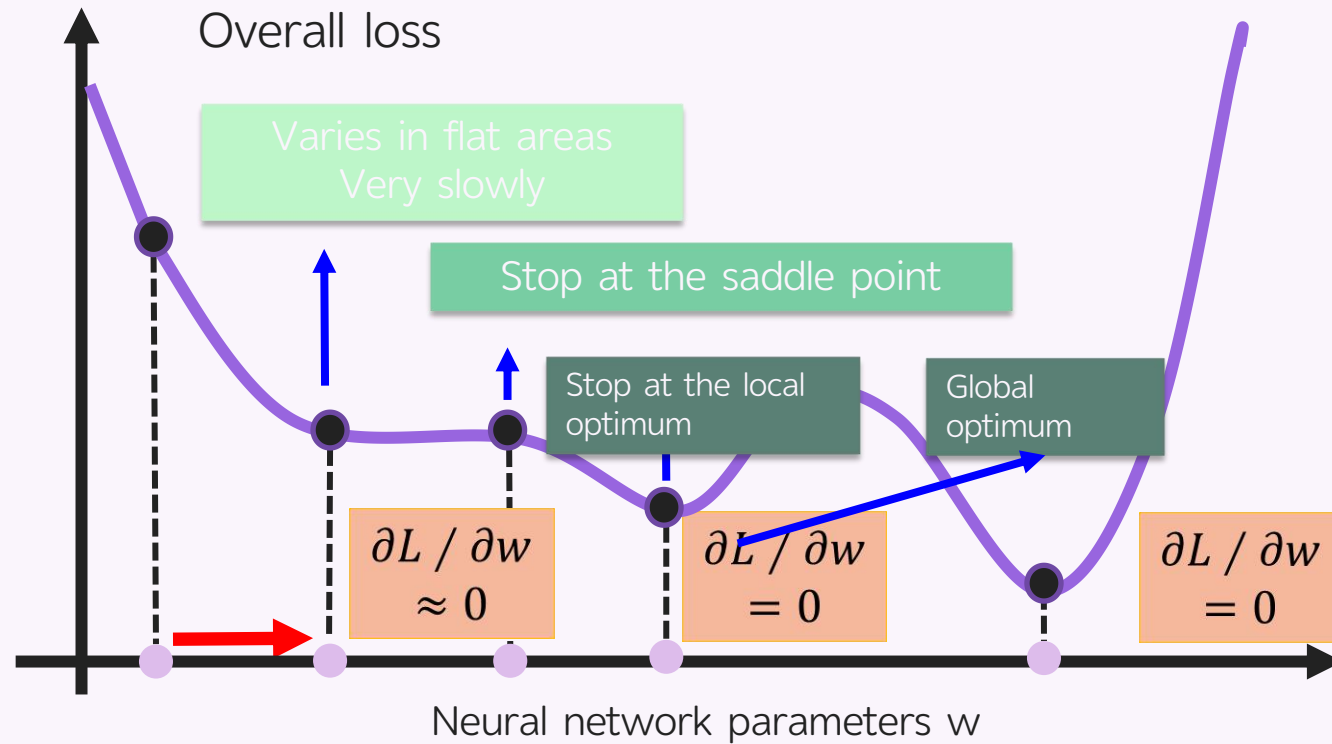
Update

$$\omega = \omega - \alpha \frac{\partial cost}{\partial \omega}$$

Update

$$\omega = \omega - \alpha \frac{1}{N} \sum_{n=1}^N 2 \cdot x_n \cdot (x_n \cdot \omega - y_n)$$

Gradient descent optimization



Characteristics of stochastic gradient descent

- **Efficiency:** Random gradient descent significantly reduces the computational cost of each update by considering only a single sample or a subset of random samples to calculate gradients, making it particularly suitable for large-scale datasets and achieving efficient optimization.
- **Volatility:** The optimization process of stochastic gradient descent often exhibits high volatility, as each update is based on gradient information from only one or a small number of samples, resulting in the optimization path jumping on the surface of the global loss function.
- **Local convergence:** Although the optimization process of stochastic gradient descent has fluctuations, it can usually quickly converge to the local optimal solution or its vicinity. However, this does not mean that the algorithm can stably find the global optimal solution.
- **Adaptability:** Stochastic gradient descent has good adaptability to optimization problems of non convex functions and functions with multiple local optima. Due to the fluctuation of the optimization path of the algorithm, it is possible to avoid getting stuck in local optima.

Advantages of small batch gradient descent

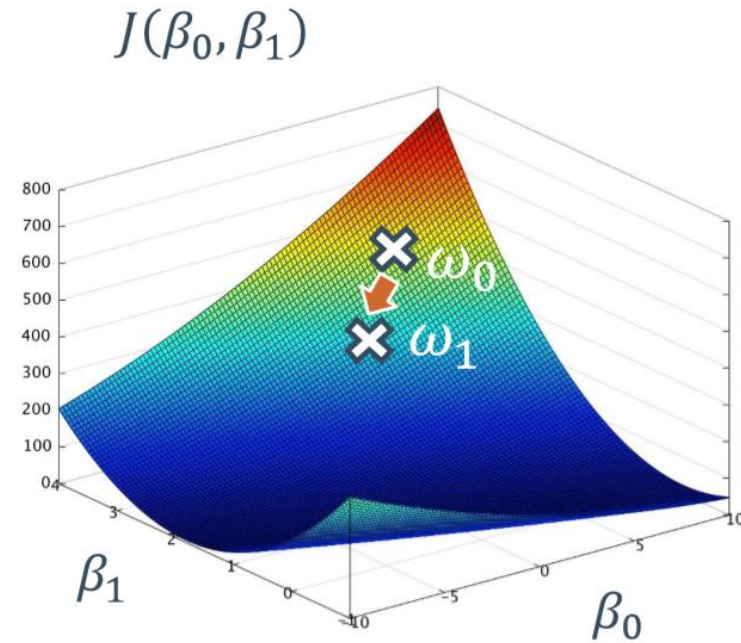
- **Efficient and stable coexistence:** Small batch gradient descent calculates gradients by considering a fixed number of sample subsets, achieving efficient optimization while reducing random fluctuations and providing a more stable optimization process than stochastic gradient descent.
- **Flexible parameter tuning:** Small batch gradient descent allows users to balance computing resources and optimize efficiency by adjusting batch sizes. A smaller batch size can reduce computational resource requirements, while a larger batch size can improve optimization stability.
- **Easy to implement:** The small batch gradient descent algorithm is easy to implement in practical applications and can update parameters through simple loops and matrix operations. This concise and efficient implementation makes it an ideal choice for handling medium-sized datasets.
- **Suitable for multiple frameworks:** The small batch gradient descent algorithm can be easily combined with various machine learning frameworks and libraries such as TensorFlow, PyTorch, etc. These frameworks typically provide optimized mathematical operations and automatic differentiation tools.

Batch Gradient Descent

- Then use the gradient (∇) and the cost function to calculate the next point (ω_1) from the current one (ω_0):

$$\omega_1 = \omega_0 - \alpha \nabla \frac{1}{2} \sum_{i=1}^m \left((\beta_0 + \beta_1 x_{obs}^{(i)}) - y_{obs}^{(i)} \right)^2$$

- The learning rate (α) is a tunable parameter that determines step size



Stochastic Gradient Descent

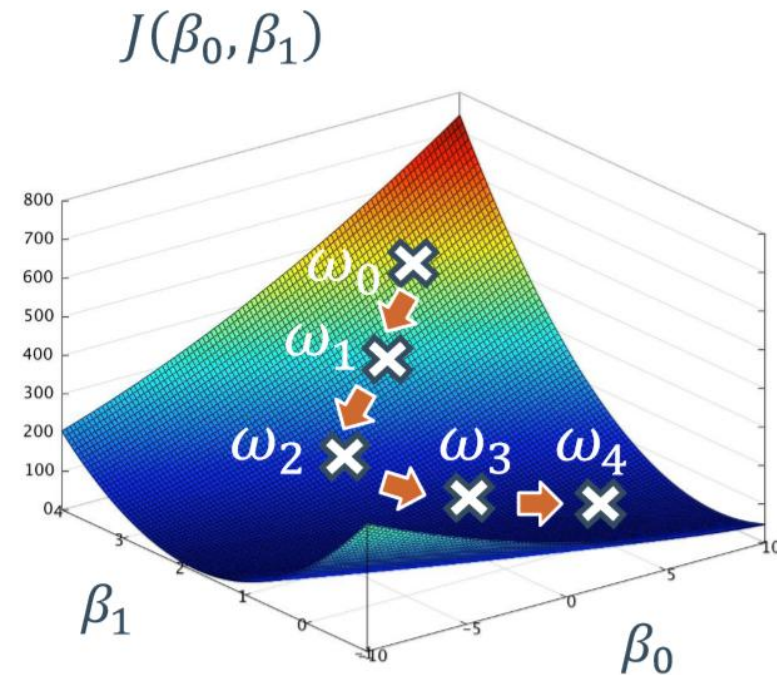
- Use a single data point to determine the gradient and cost function instead of all the data

$$\omega_1 = \omega_0 - \alpha \nabla \frac{1}{2} \left((\beta_0 + \beta_1 x_{obs}^{(0)}) - y_{obs}^{(0)} \right)^2$$

...

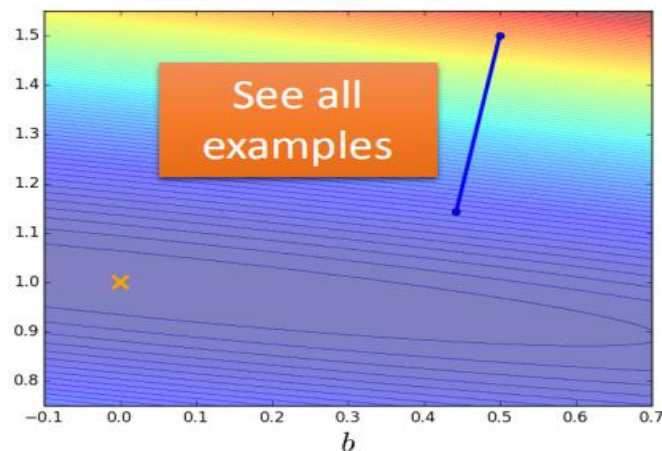
$$\omega_4 = \omega_3 - \alpha \nabla \frac{1}{2} \left((\beta_0 + \beta_1 x_{obs}^{(3)}) - y_{obs}^{(3)} \right)^2$$

- Path is less direct due to noise in single data point—"stochastic"



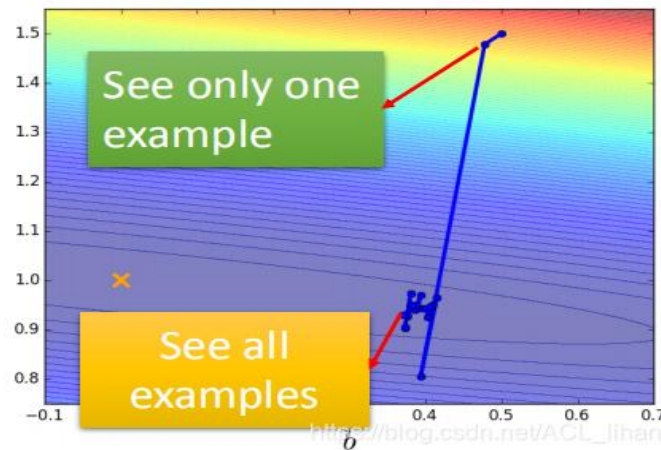
Gradient Descent

Update after seeing all examples



Stochastic Gradient Descent

Update for each example
If there are 20 examples,
20 times faster.



$$L = \sum_n \left(\hat{y}^n - \left(b + \sum w_i x_i^n \right) \right)^2$$

Loss is the summation over all training examples

◆ Gradient Descent $\theta^i = \theta^{i-1} - \eta \nabla L(\theta^{i-1})$

◆ Stochastic Gradient Descent

Faster!

Pick an example x^n

$$L^n = \left(\hat{y}^n - \left(b + \sum w_i x_i^n \right) \right)^2$$

$$\theta^i = \theta^{i-1} - \eta \nabla L^n(\theta^{i-1})$$

Loss for only one example

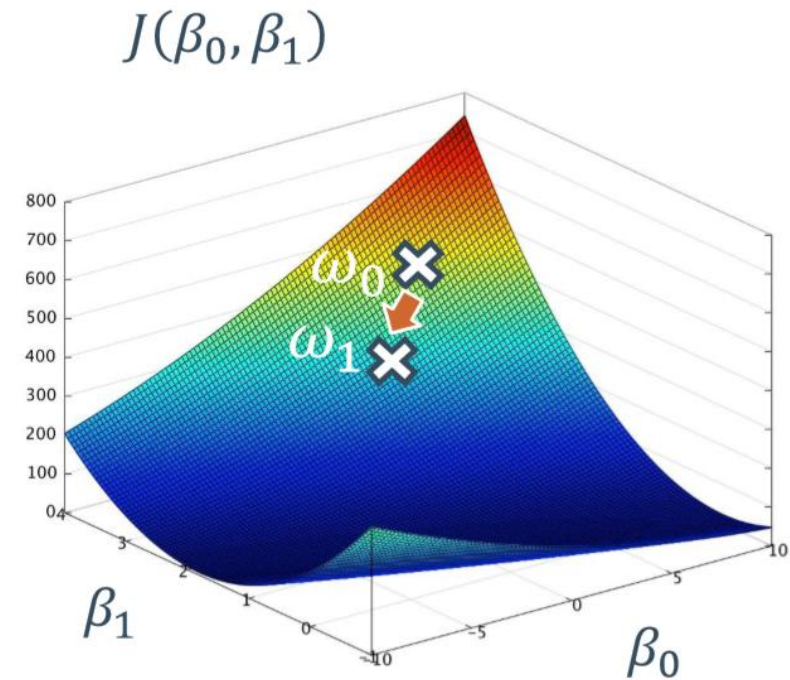
Small batch gradient descent

- Perform an update for every n training examples

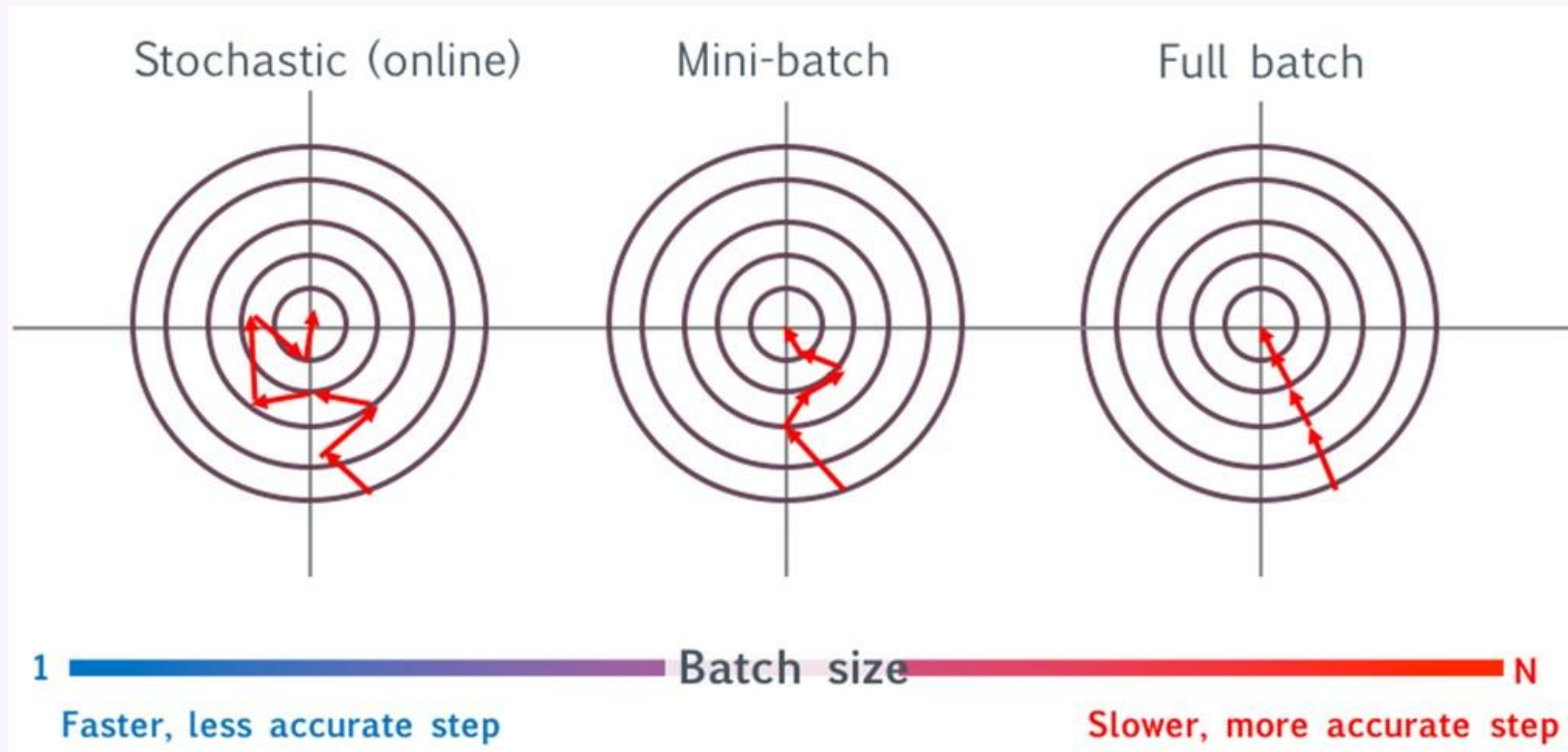
$$\omega_1 = \omega_0 - \alpha \nabla \frac{1}{2} \sum_{i=1}^n \left((\beta_0 + \beta_1 x_{obs}^{(i)}) - y_{obs}^{(i)} \right)^2$$

Best of both worlds:

- Reduced memory relative to "vanilla" gradient descent
- Less noisy than stochastic gradient descent



Batch size comparison



Batch size impact analysis

- **Batch size and memory usage:** Increasing batch size can improve memory usage efficiency as less data needs to be stored in memory. This is particularly important when dealing with large-scale datasets or training models in resource constrained environments.
- **Batch size and gradient quality:** Smaller batch sizes typically provide more refined gradient information, helping algorithms escape from local optima. However, larger batch sizes may make the optimization process too smooth, resulting in slower convergence speed.
- **Batch size and parallelization:** For platforms that support parallel computing, such as GPUs and distributed systems, increasing batch size appropriately can more effectively utilize computing resources, improve optimized parallel processing capabilities, and overall efficiency.
- **Batch size and model generalization:** In some cases, training with smaller batch sizes may lead to a decrease in the model's generalization ability on test data. This may be because small batch gradient descent is prone to getting stuck in local optima.

03

Gradient descent optimization algorithm

Principle and Implementation of Momentum Method



01

Momentum philosophy

Based on gradient descent, historical gradient information is introduced as a reference for the current descent direction, thereby accelerating convergence and reducing gradient oscillations.

02

Implementation strategy

Maintain a velocity vector opposite to the current gradient, which gradually accumulates historical gradient information during the iteration process and affects the direction of updating the current weights.

03

Effect improvement

By introducing Momentum, the optimizer can accelerate SGD in the relevant direction, effectively suppress gradient oscillation, and is particularly suitable for optimization problems with non-stationary objectives.

Comparison between SGD and Momentum

Applicable scenarios

SGD is suitable for small-scale datasets and simple problems, while Momentum is more suitable for large-scale datasets and complex problems, which can effectively improve optimization efficiency.

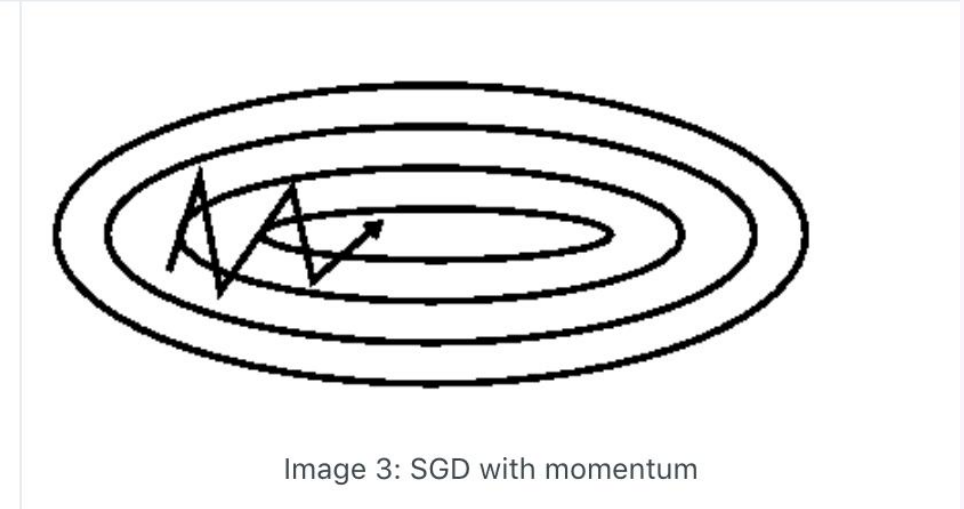
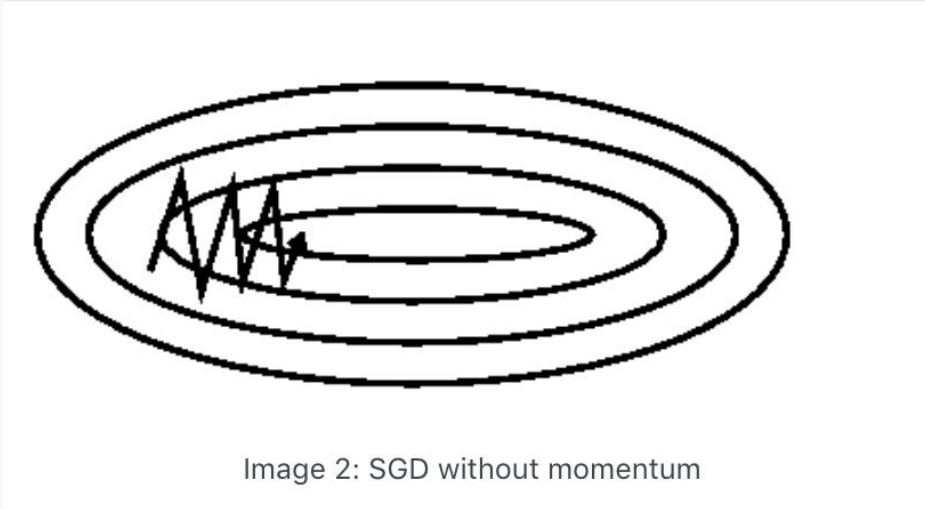
Convergence speed

Under the same conditions, Momentum usually converges faster than SGD because it can find the optimal solution faster, thereby reducing the number of iterations.

Breaking out of local optima

Momentum introduces historical gradient information to better break out of local optima and find solutions closer to global optima, thereby improving optimization performance.

Momentum

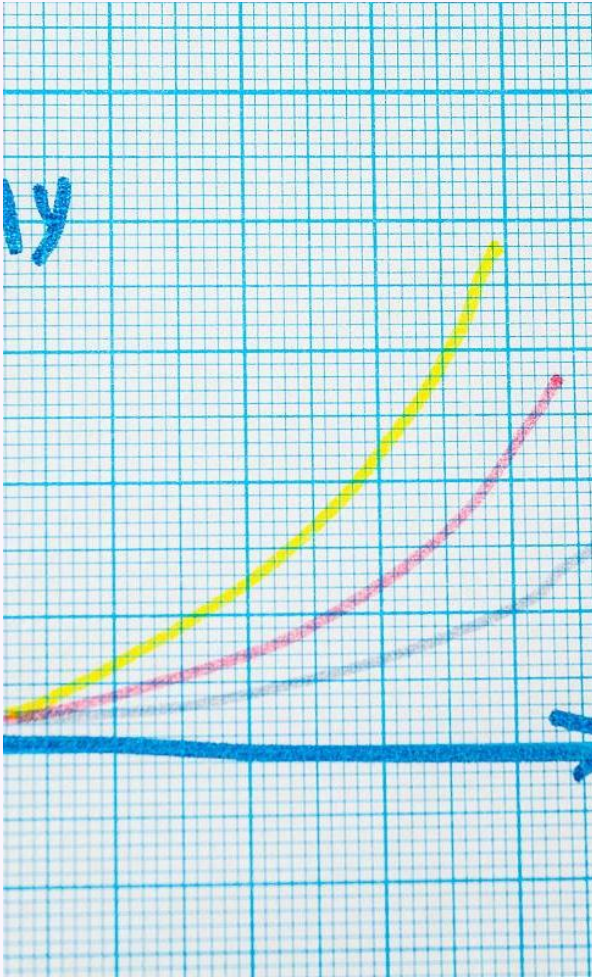


- The momentum method adds the components of the previous update v to the current update according to the following formula: The momentum term γ is typically set to 0.9.

$$v_t = \gamma v_{t-1} + \eta \nabla_{\theta} J(\theta) \quad (1)$$

$$\theta = \theta - v_t \quad (2)$$

Adaptive learning rate method



Learning rate adjustment

dynamically adjust the learning rate based on changes in losses during the training process, reducing the learning rate in flat areas to avoid oscillations, and increasing the learning rate in steep areas to accelerate the decline.

Application advantages

The adaptive learning rate method can automatically process data of different scales, improve the stability and efficiency of the training process, and reduce the need for manual intervention.

Limitations of the method

It is necessary to carefully choose the initial learning rate and monitor changes in the learning rate during the training process to avoid situations of excessive adjustment or insufficient updates.

Method	Key Features	Applications
Momentum	Accelerates SGD, reduces oscillations, good for non-smooth gradients	General optimization
NAG	Predicts updates, improves responsiveness, prevents overshooting	RNN tasks
Adagrad	Handles sparse data well	Deep learning with sparse data
Adadelta	No need to set learning rate manually	Adaptive learning with varying gradient scales
RMSprop	Adaptive learning rate (special Adadelta case)	General optimization, non-stationary objectives
Adam	Combines sparse gradient handling (Adagrad) and non-stationary support (Momentum)	Wide applications, efficient convergence
AdaMax	Adam update	Similar to Adam, potential high performance
Nadam	Combines RMSprop and momentum advantages	General optimization
Lookahead	From Hinton's team, newer development	Potential optimization improvements

Adagrad

$$\eta^t = \frac{\eta}{\sqrt{t+1}} \quad g^t = \frac{\partial L(\theta^t)}{\partial w}$$

- Divide the learning rate of each parameter by the ***root mean square of its previous derivatives***

Vanilla Gradient descent

$$w^{t+1} \leftarrow w^t - \eta^t g^t$$

w is one parameters

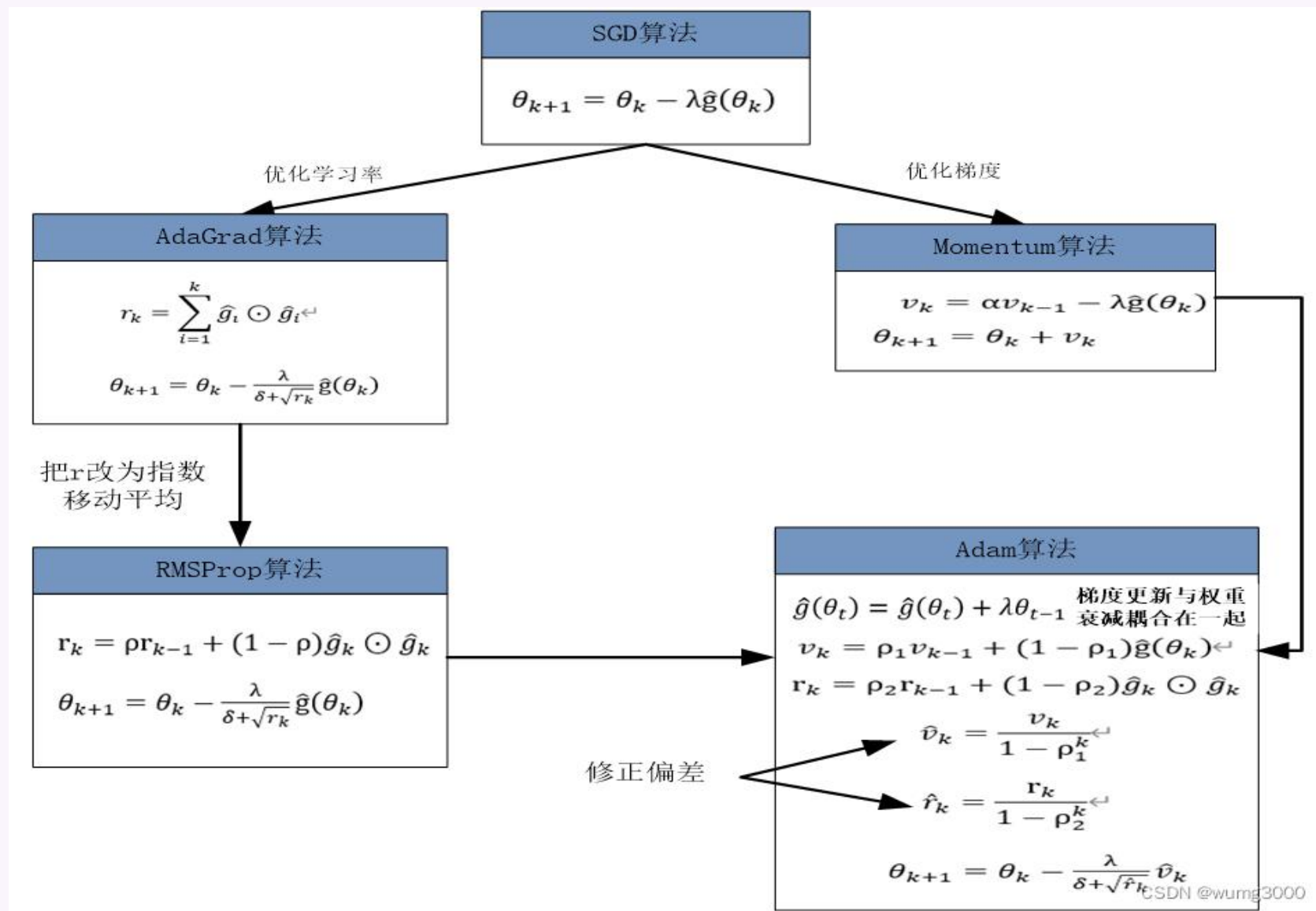
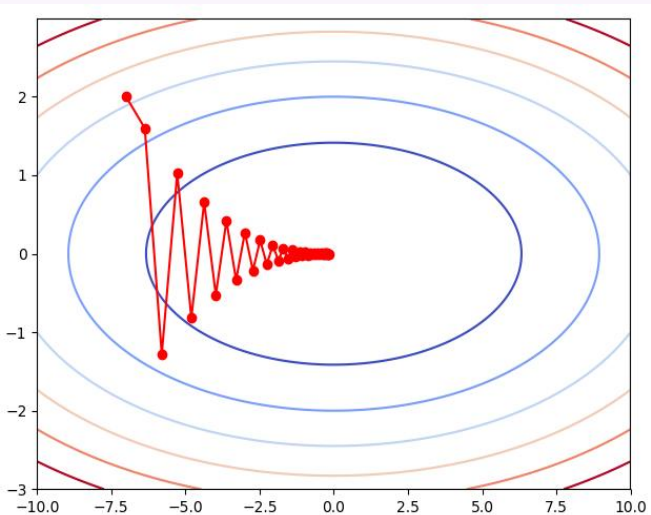
Adagrad

$$w^{t+1} \leftarrow w^t - \frac{\eta^t}{\sigma^t} g^t$$

σ^t : ***root mean square*** of the previous derivatives of parameter w

Parameter dependent

Adam



Python implementation of gradient descent

Random Gradient Descent

Random gradient descent optimizes the loss function by iteratively calculating gradients and updating weights for individual data points. This process reduces the demand for computing resources and is suitable for large datasets.

Small batch gradient descent

The small batch gradient descent algorithm calculates gradients by processing a fixed number of data points (known as batch size), balancing computational efficiency and resource utilization, and is suitable for datasets of various sizes.

Batch gradient descent

The batch gradient descent algorithm considers the entire dataset when calculating gradients, thus providing more accurate gradient estimates. This helps the algorithm to converge better to the optimal solution, but may increase computational resource requirements.



04

Prevent overfitting techniques

Principle of Dropout Mechanism

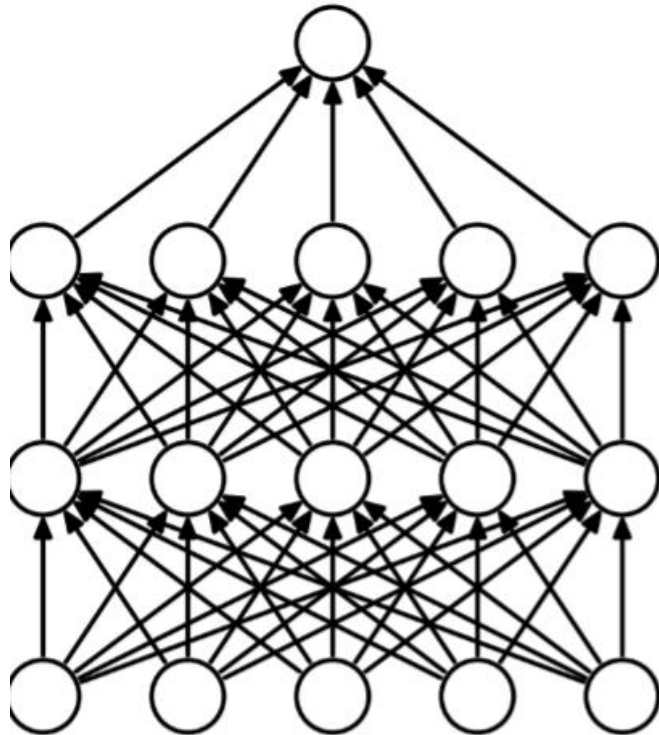
Dropout principle

Dropout mechanism randomly discards some neurons during neural network training, effectively preventing overfitting and improving model generalization ability. It is a commonly used regularization technique in deep learning.

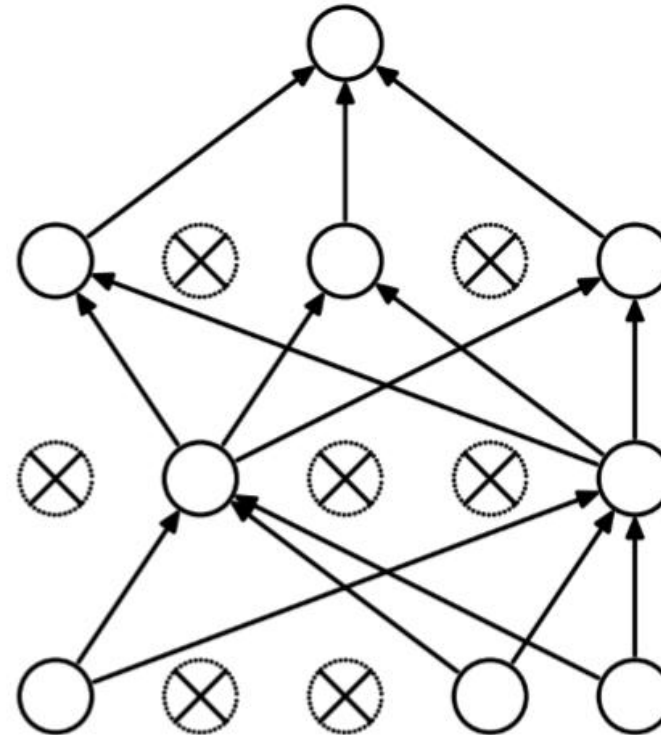
Implementation method

By randomly setting some input units to 0 during forward propagation, the nn. Dropout layer implements the Dropout mechanism, forcing the network to relearn each time it passes, effectively preventing overfitting.

Dropout



(a) Standard Neural Net



(b) After applying dropout

Principle of Dropout Mechanism

01

Scale the coefficients of the training performance (dropout rate) to maintain the same expected output level.

02

Inference performance: During the inference phase (i.e. model evaluation or prediction phase), the Dropout layer is disabled to ensure that input data is not affected, thereby ensuring stable and reliable model performance.

Application scenarios of early stopping method

- **Monitoring performance:** During the training process, closely monitor changes in model performance, including metrics such as accuracy and recall. Once performance begins to decline, stop training immediately.
- **Adjustment strategy:** Adjust the training strategy in a timely manner based on the monitoring results. For example, adjusting the learning rate, adding regularization terms, etc. To alleviate the occurrence and development of overfitting phenomenon.
- **Application scenario:** Early stopping method is applicable to various machine learning models and datasets. It is particularly commonly used in neural network training and can effectively prevent the occurrence and development of overfitting phenomena.
- **Combination method:** Use in combination with other regularization methods. By comprehensively applying multiple regularization techniques, the occurrence and development of overfitting phenomena can be effectively alleviated to enhance the generalization ability of the model.

Dropout code – Pytorch example

```
1. import torch.nn as nn
2. class MyModel(nn.Module):
3.     def __init__(self, input_size, hidden_size, output_size,
4. dropout_rate=0.5):
5.         super(MyModel, self).__init__()
6.         self.fc1 = nn.Linear(input_size, hidden_size)
7.         self.dropout1 = nn.Dropout(dropout_rate)
8.         self.fc2 = nn.Linear(hidden_size, hidden_size)
9.         self.dropout2 = nn.Dropout(dropout_rate)
10.        self.fc3 = nn.Linear(hidden_size, output_size)
11.    def forward(self, x):
12.        x = self.fc1(x)
13.        x = nn.ReLU()(x)
14.        x = self.dropout1(x)
15.        x = self.fc2(x)
16.        x = nn.ReLU()(x)
17.        x = self.dropout2(x)
18.        x = self.fc3(x)
19.        return x
```

```
1. model = MyModel(input_size=60, hidden_size=30,
2. output_size=1, dropout_rate=0.2)
3.
4. # Training loop
5. for epoch in range(num_epochs):
6.     # Forward pass
7.     outputs = model(X_train)
8.
9.     # Backward pass and optimization
10.    loss = criterion(outputs, y_train)
11.    optimizer.zero_grad()
12.    loss.backward()
13.    optimizer.step()
14.
15. # Evaluation
16. model.eval()
17. with torch.no_grad():
18.     test_outputs = model(X_test)
```

Core Function: During training, randomly sets some input units to 0, reducing excessive reliance between neurons and thereby alleviating overfitting.

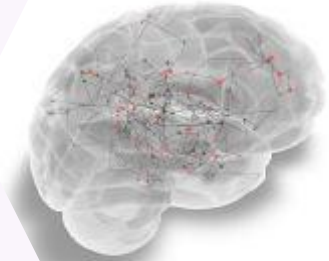
Training Phase:

- Randomly zeros out input elements with a probability specified by `dropout_rate`.
- To maintain the expected magnitude of the output (preventing an overall decrease due to some elements being zeroed), the output is scaled by a factor of $1/(1 - \text{dropout_rate})$.

Inference Phase:

- After calling `model.eval()`, the dropout layer is deactivated, and inputs pass through unchanged (no zeroing or scaling operations are performed).

Dropout Implementation Method

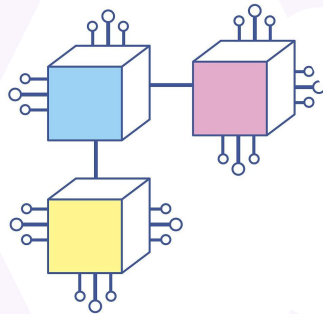
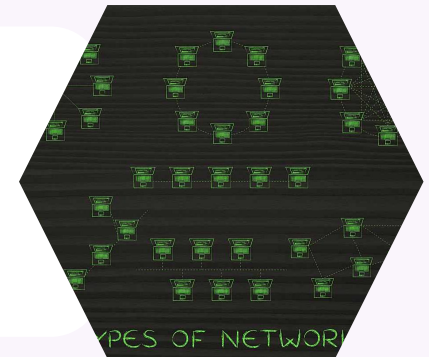


Dropout Introduction

Dropout is a regularization technique that prevents overfitting in neural networks by randomly discarding a certain proportion of neurons to reduce model complexity and effectively avoid overfitting.

Dropout implementation

During neural network training, the Dropout layer sets the input unit to 0 with a specified probability to achieve random dropout. This mechanism forces the network to learn more robust feature representations.



Dropout effect

Dropout technology introduces randomness to enable neural networks to learn more robust features during training, significantly improving the model's generalization ability. It is a key technique in deep learning.

05

Regression Models and Regularization

Overview of Regularization Methods

- **Regularization in ML:** Prevent overfitting and improve model generalization. By constraining the complexity of the model, reducing its dependence on training data, and improving its predictive ability for new data.
- **L1 and L2 regularization:** L1 regularization penalizes sparse coefficients, promotes model sparsity, and effectively prevents overfitting. L2 regularization achieves overfitting prevention by punishing the sum of squares and stabilizing the coefficients.
- **Dropout mechanism:** During the training process of neural networks, randomly discarding some neurons effectively prevents overfitting and improves the model's generalization ability. It is a commonly used regularization technique in deep learning.
- **Early stopping method:** During the training process, closely monitor the performance of the model. Once performance begins to decline, stop training immediately to prevent overfitting and ensure that the model maintains good generalization ability.

Fundamentals of Linear Regression

- **Performance metric:** Mean Square Error (MSE) measures the average of the sum of squares of the difference between the predicted value and the true value of the model, focusing on the accuracy of the model's predictions.
- **Regularization technique:** Adding penalty terms to the cost function to limit model complexity, effectively prevent overfitting, and improve model generalization ability.
- **Gradient descent:** By calculating the gradient of the cost function on the weights and updating the weights in the opposite direction of the gradient, the cost function is minimized.
- **Limitations of linear regression:** The assumption of linear relationship may be overly simplified, and real-world data often exhibits nonlinear characteristics, which affects the fitting effect of the model.

Ridge regression characteristics

01

L2 regularization

Ridge regression uses L2 regularization techniques to effectively limit model complexity and prevent overfitting by penalizing the square of the coefficient.

02

Stability advantage

Compared to Lasso regression, Ridge regression is more stable and less susceptible to small changes in data, making its prediction results more reliable.

03

Multicollinearity processing

Ridge regression can efficiently handle multicollinearity problems by introducing penalty terms to effectively alleviate the impact of collinearity on the model.

04

Limitations

Ridge regression performs poorly in feature selection, making it difficult to directly eliminate unimportant features, which may lead to high model complexity.

Lasso regression characteristics

L1 regularization

Lasso regression uses L1 regularization technique to effectively promote model sparsity and facilitate feature selection by penalizing the absolute values of coefficients.

Sparsity advantage

The sparsity property of Lasso regression not only helps simplify the model, but also reveals the importance of features and improves the interpretability of the model.

Handling multicollinearity

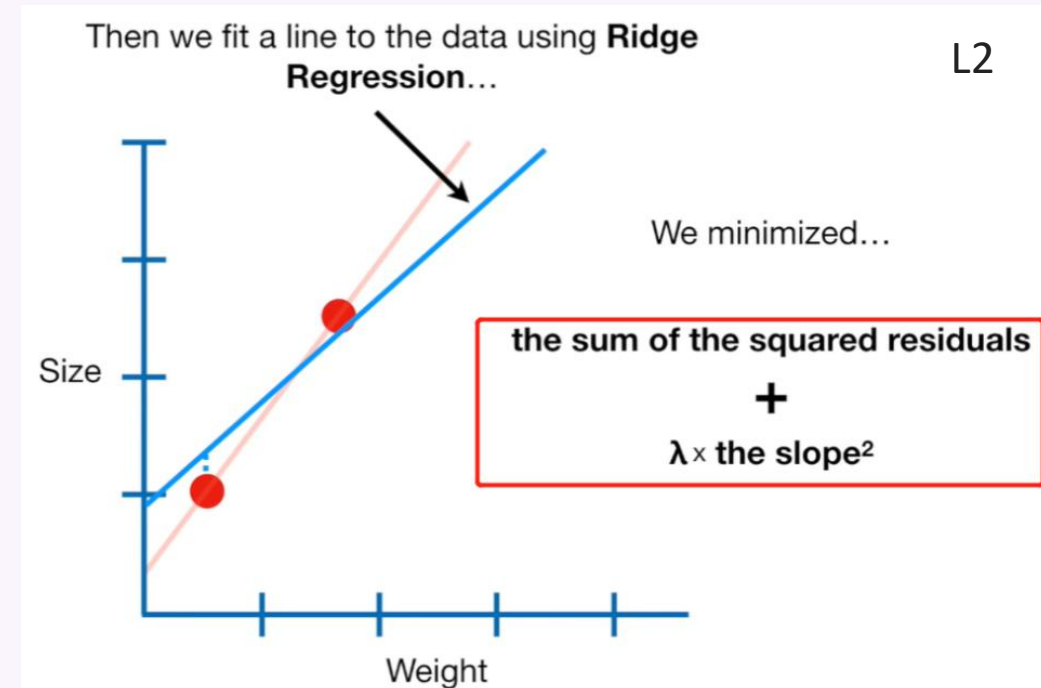
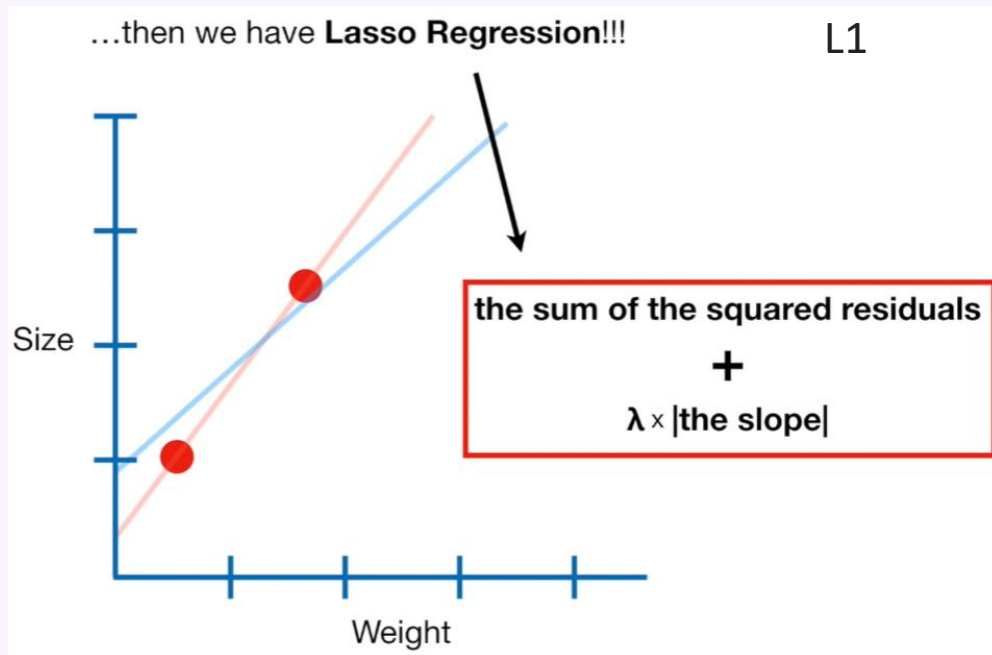
Lasso regression can handle data with multicollinearity by automatically selecting important features and reducing the impact of collinearity.

Limitations

Lasso regression may perform poorly when the number of features far exceeds the sample size, leading to inaccurate coefficient estimation due to excessive punishment.

Lasso vs Ridge

- If the model contains a large number of irrelevant variables, Lasso regression can exclude these irrelevant variables, making it superior to ridge regression.
- Conversely, if most variables in the model are relevant, ridge regression will not incorrectly eliminate any variables, thus being superior to Lasso regression.



Comparison of L1 and L2 Regularization

- **L1 regularization:** With its unique sparsity penalty mechanism, L1 regularization can automatically perform feature selection, reduce the coefficients of unimportant features to zero, enhance the model's generalization ability, and is suitable for feature selection tasks.
- **L2 regularization:** L2 regularization uses sum of squares penalty to stabilize model coefficients, effectively prevent overfitting, ensure model generalization ability, and is suitable for scenarios that require stable model training and want to avoid overfitting.
- **Selection strategy:** When choosing a regularization method, it is necessary to balance sparsity and stability requirements. L1 regularization is suitable for situations where you want to automatically select important features and tolerate some non important feature noise.
- **Combining the advantages of L1 and L2 regularization,** adopting Elastic Net Regression to flexibly respond to different data structures and requirements, and improving model performance and generalization ability.

Elastic Network Regression

L1+L2 regularization

Elastic Net regression combines L1 and L2 regularization techniques, while penalizing the absolute and square values of coefficients, to achieve flexible feature selection.

Flexibility

The flexibility of Elastic Net regression stems from its ability to simultaneously utilize the advantages of L1 and L2 regularization, promoting feature sparsity while maintaining model stability.

Application scenario

Elastic Net regression is suitable for situations with a large number of features, and improves model prediction performance by effectively handling multicollinearity.

Limitations

When the number of features far exceeds the number of samples, Elastic Net regression may perform poorly due to inaccurate coefficient estimation caused by excessive punishment.

Regularization code example

Lasso regression

Lasso regression penalizes coefficients by introducing L1 regularization terms, encouraging the model to perform feature selection by selectively reducing certain coefficients to zero, in order to achieve sparsity and more efficient feature utilization.

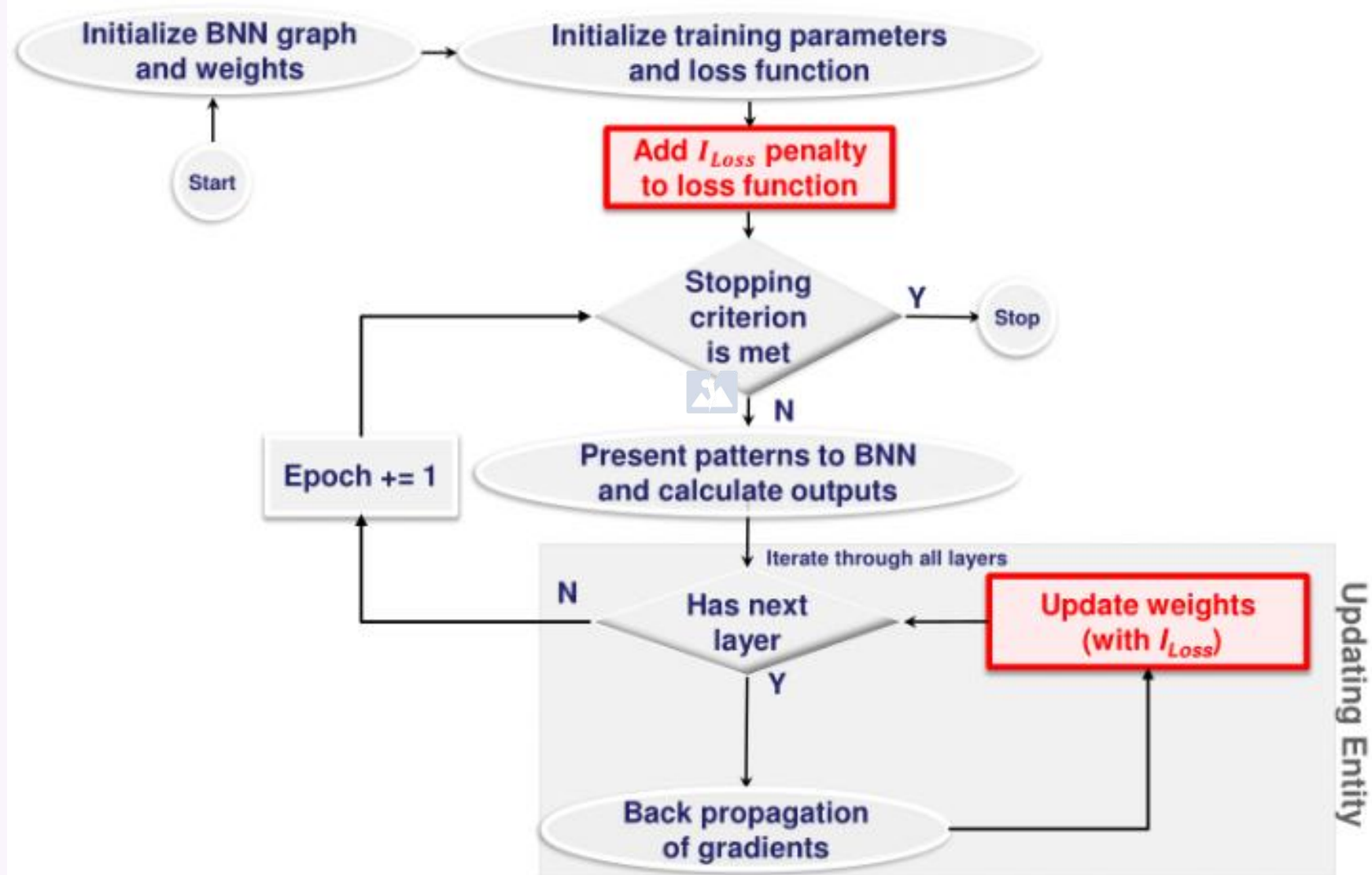
Ridge regression

Ridge regression uses L2 regularization term to penalize the squared coefficients of the model to prevent overfitting. This regularization method helps stabilize the model, causing the coefficients to approach zero but remain non-zero.

Elastic Net

Elastic Net regression combines L1 and L2 regularization to effectively handle multicollinearity problems by simultaneously penalizing the absolute and square values of coefficients, while promoting model sparsity and stability.

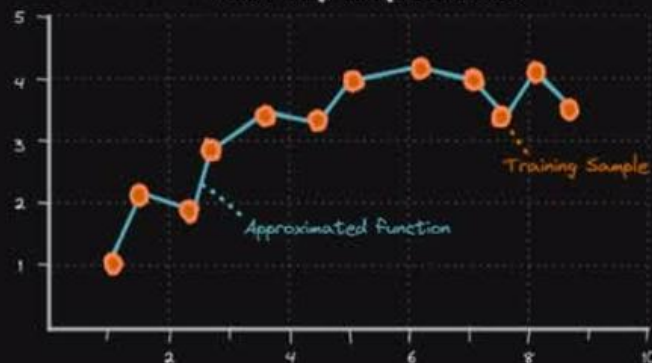
Regularization in Binary Neural Network



L2 REGULARIZATION

Function Approximated by Network
to Fit the Training Set

Too Complex of a Function
(Overfitting, not generalizable)

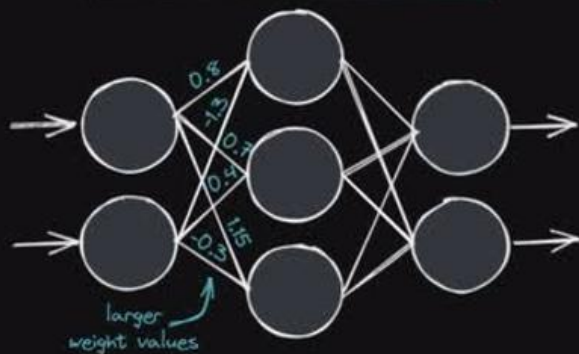


More acceptable function
(Smooth, more generalizable)

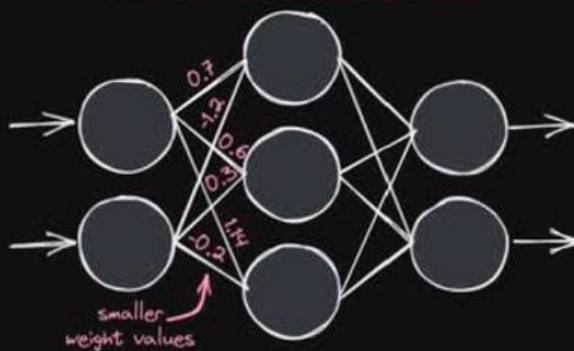


L2 Regularization
Larger Weights Penalize Loss for Complexity

More Complex Model, Larger Penalty



Less Complex Model, Less Penalty



loss = loss term + regularization term

$$\lambda \sum_{j=0}^n \|w_j\|^2$$



06

Neural network evaluation indicators

Construction of confusion matrix

1

Overview of confusion matrix

Confusion matrix is a specific data presentation method used to evaluate the performance of classification models, especially in binary or multi classification problems.

2

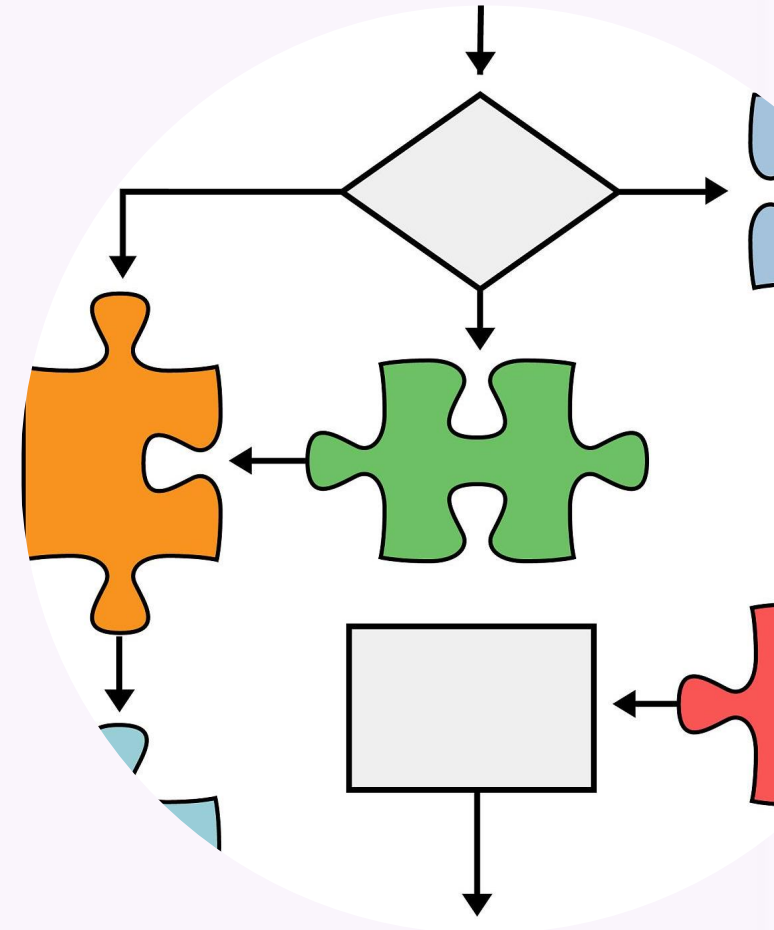
Matrix rows and columns

Each row of the confusion matrix represents the actual category, and each column represents the predicted category. Through the matrix, the performance of the model on different categories can be visually observed.

3

The meaning of matrix elements

Each element in the matrix represents the combination of the predicted results of the model on the corresponding category and the actual category, and is a key data for performance evaluation.



Accuracy and recall rate

01

Accuracy definition

Accuracy is an indicator that measures the correctness of a model's predictions, that is, the number of correct predictions divided by the total number of predictions. It provides an overview of the overall prediction accuracy of the model.

02

Recall rate definition

Recall rate, also known as sensitivity, is an important indicator for evaluating a model's ability to identify positive categories (such as signals, targets, etc.). It reflects the degree of recognition of positive classes by the model.

03

Accuracy and recall

Both are important indicators for evaluating model performance, but there is a trade-off between them. Therefore, when evaluating the model, it is necessary to consider both accuracy and recall comprehensively.

ROC curve analysis



Principle of ROC curve

The ROC curve is plotted with true positive rate (sensitivity) as the y-axis and false positive rate as the x-axis. By plotting the curve, the classification performance of the model can be intuitively reflected.

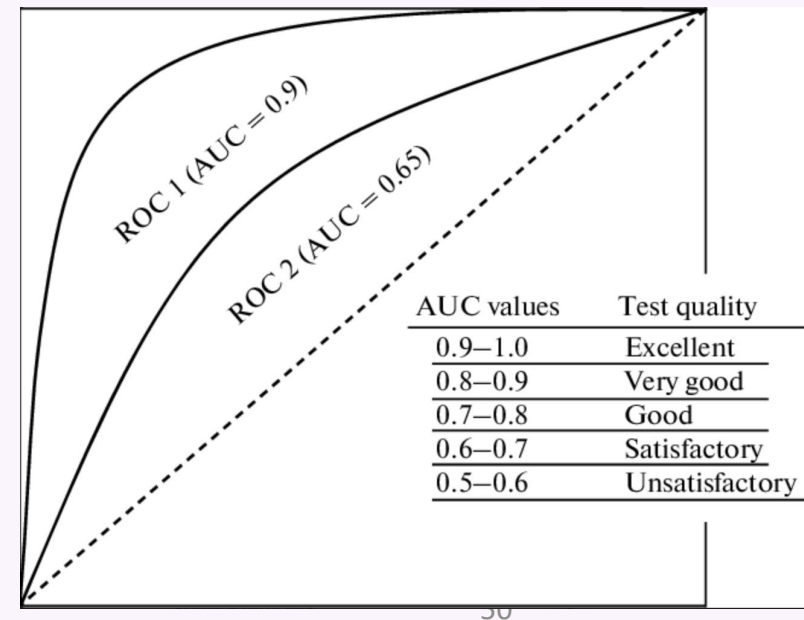
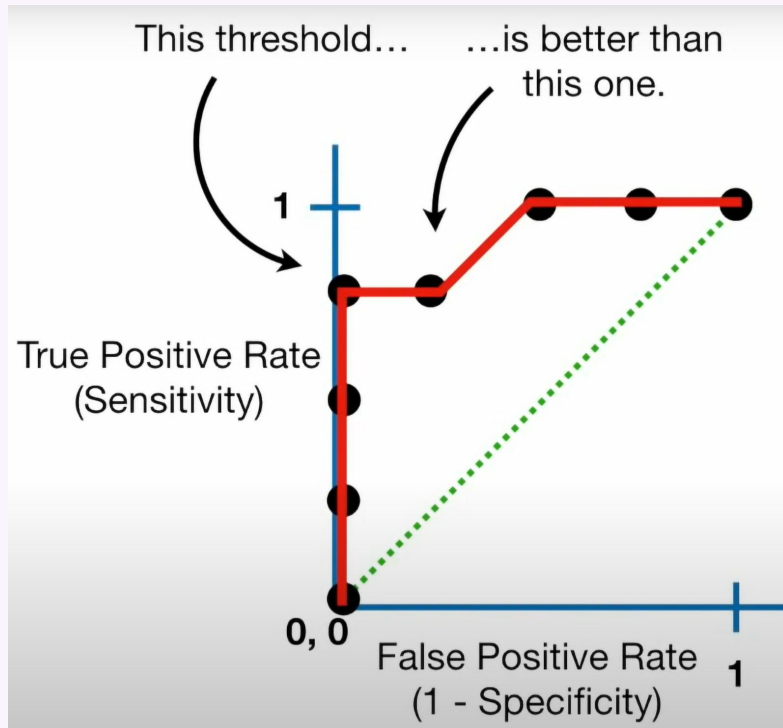
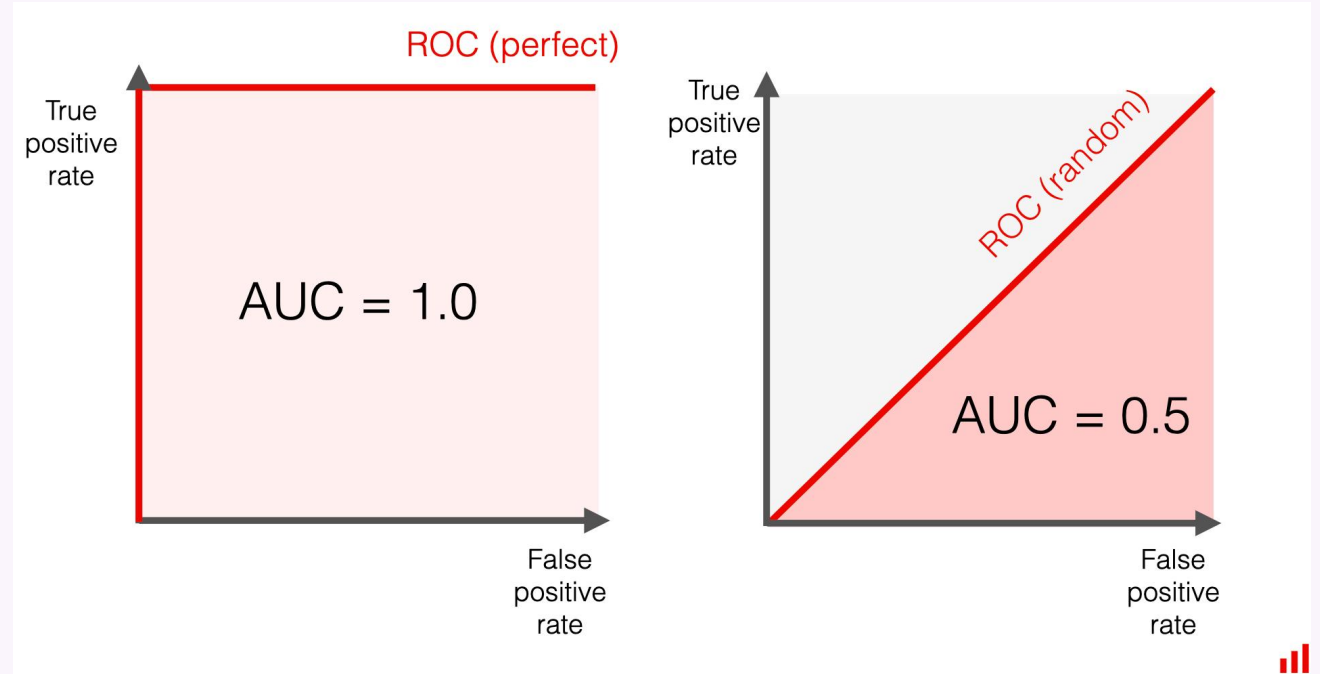
Area under the curve

AUC value is the area under the ROC curve, which quantitatively evaluates the classification performance of the model. The larger the AUC value, the better the classification performance of the model.

Curve and threshold adjustment

By adjusting the threshold of the classifier, the shape of the ROC curve and the size of the AUC value can be influenced. Therefore, choosing the appropriate threshold is crucial for improving model performance.

ROC and AUC



AUC value calculation



AUC value definition

AUC value refers to the area under the ROC curve, used to quantify the classification performance of the model. The numerical range is from 0 to 1, and the larger the AUC value, the better the classification performance of the model.



Calculation method

There are multiple methods available for calculating AUC values, including trapezoidal numerical integration method, approximation method (such as Mann Whitney U statistic), and other statistical methods.



Model evaluation reference

In binary or multi class classification problems, AUC value can be used as one of the reference criteria for evaluating model performance. By combining other indicators, the performance of the model can be evaluated more comprehensively.

Calculation of evaluation indicators

Accuracy



Accuracy measures the accuracy of a model in predicting positive samples, that is, the ratio of the number of truly predicted positive samples to the number of all predicted positive samples by the model. High precision indicates that the model performs well in identifying positive samples.

Recall rate



Recall rate (also known as sensitivity) is an indicator used to evaluate a model's ability to recognize positive samples. It represents the proportion of instances that are actually positive samples that the model correctly recognizes as positive samples. A high recall rate means that the model can identify more positive samples.

Code Example : Numpy MLP Optimization

- Core Result: 97.90% test accuracy achieved via comprehensive optimization.
- 1. Key Technical Improvements
 - Regularization: L2 (weight decay) + Dropout (0.25)
 - Training Stability: Improved BatchNorm, momentum SGD, gradient clipping
 - Efficiency: Adaptive LR (warm-up + cosine decay), early stopping (patience=8)
 - Utility: Training history recording, dedicated evaluate method, ELU support
- 2. Optimized Model Architecture
 - Structure: 784 (input) → 256 → 128 → 64 → 32 → 10 (output)
 - Activation: Leaky ReLU (prevents neuron death)
 - Optimizer: Adam
 - Training Config: Batch size 128, random train/val split
- 3. Performance & Files
 - Metrics: 97.90% test acc, 97.88% best val acc (up from initial 58.31%)
 - File Update: mlp_numpy.py (MLP class, training flow, evaluation outputs)

Code Example : Pytorch MLP Optimization

- Core Result: The optimized model achieved a test accuracy of 98.46% — a significant increase from the original version's 97.84%.
- 1. Enhanced Network Architecture
 - Expanded to 4 hidden layers (512, 256, 128, 64 neurons)
 - Added batch normalization to each layer for improved training stability
 - Adopted the optimized Xavier weight initialization method
- 2. Data Augmentation
 - Random small-angle rotation (± 5 degrees)
 - Small-range translation operations
 - Strengthened the model's generalization capability
- 3. Advanced Training Strategies
 - AdamW Optimizer: Combines Adam's adaptive learning rate with L2 weight decay
 - Learning Rate Scheduler: Dynamically reduces learning rate based on validation loss
 - Early Stopping: Prevents overfitting and automatically selects the best model
 - Gradient Clipping: Mitigates gradient explosion issues
 - Layer-wise Dropout: Implements a more rational regularization strategy
- 4. Comprehensive Analysis Tools
 - Visualization of mispredictions
 - Confusion matrix analysis
 - Detailed training history monitoring (loss, accuracy, learning rate changes)